UNIVERSITÉ
PARIS DESCARTES
UFR DE MATHÉMATIQUES ET INFORMATIQUE

Documentation interne

Calcul de relations spatiales directionnelles à partir de l'histogramme d'angle

UNIVERSITÉ PARIS DESCARTES
UFR DE MATHÉMATIQUES ET INFORMATIQUE

Documentation interne

Calcul de relations spatiales directionnelles à partir de l'histogramme d'angles

Les informations d'identification du document

Référence du document :	D7
Version du document :	1.00
Date du document :	19 avril 2023
Auteur(s) :	Damien Devasenaradjounay agar Rayan Al Mohaize Rolland Xie

Les éléments de vérification du document

Validé par :	M.Wendling
Validé le :	21/04/2023
Soumis le :	21/04/2023
Type de diffusion :	Document électronique (.pdf)
Confidentialité :	Standard / Étudiants UFR Maths-Info de l'université Paris Descartes

Mots clés : Modules ; Histogramme ; Barycentre ; Interface ; ligne de commande ; Package ; Modules

Sommaire

Sommaire 3

1. Introduction (ou préambule) 5

1.1. Objectifs et méthodes 5

1.2. Cadre 5

1.3. Documents de référence 6

2. Guide de lecture 6

2.1. Maîtrise d'œuvre 6

2.2. Maîtrise d'ouvrage 7

3. Concepts de base 7

4. Relation d'utilisation 8

4.1. Lecture des arguments pour la ligne de commande avec arguments 8

4.2. Lecture d'image 9

4.3. Conversion de l'image en tableaux a 2 dimensions 11

4.4. Création des 2 Objets 12

4.5. Calculs 13

4.6. Affichages et choix 13

4.7. Traitements 14

4.8. Boucle ou fin 16

5. Liste des modules 17

5.1. Modules externes 17

5.2. Modules internes 18

6. Définitions de types 19

6.1. Définitions de classes classiques 19

6.2. Définitions de classes de test unitaire 23

7. Procédures externes 25

8. Variables externes 26

9. Glossaire 27

10. Références 28

11. Index 28

1. Introduction (ou préambule)

1.1. Objectifs et méthodes

Ce document a pour but de faciliter la maintenance de l'application par un tiers, pour le projet L2P2, intitulé : **"Calcul de relations spatiales directionnelles à partir de l'histogramme d'angles"**.

Pour cela, ce document présentera dans un premier temps le contexte dans lequel s'inscrit le projet, son objectif ainsi qu'une présentation de l'équipe. Puis, dans un second temps, la relation d'utilisation avec d'autres modules, puis nous listerons les modules utilisés par ce module et ceux utilisant ce module. Dans un troisième temps, on listera les définitions de type, les procédures ainsi que les variables visibles par les modules utilisant ce module. Finalement, ce document s'achève par les définitions ou bien des informations complémentaires qui peuvent aider le lecteur à mieux comprendre ce document.

Le développement du logiciel se fait en plusieurs parties. Commençant par une lecture d'image précise, permettant de distinguer 2 objets qui seront comparés afin d'obtenir leurs angles. Pour au final pouvoir les situer grâce à leur histogramme d'angle. À cela s'ajoutent des modules permettant de diviser une application en parties plus petites, de gérer les dépendances entre ces parties de manière efficace et de rendre l'application plus modulaire, plus robuste et plus facile à maintenir. Ainsi, cela apporte un avantage pour les programmeurs puisqu'il nous permet de contrôler les interactions entre les différentes parties de l'application. Tout cela se fait dans le langage JAVA.

1.2. Cadre

Ce projet s'inscrit dans l'UE de projet de la licence d'informatique de l'UFR de Maths-Info de l'Université de Paris. L'objectif est de faire réaliser à 3 étudiants un vaste projet en quasi-autonomie et en essayant d'approcher une gestion la plus professionnelle possible du projet.

1.3. Documents de référence

De nombreux livrables ont été rendus tout au long du projet. Ces derniers ont tous été publiés sur la forge du projet et sont accessibles à tout moment. Certains participent grâce à leurs contenus, à l'élaboration de ce document ou permettent à la compréhension de ce document. Ces documents sont :

- Cahier des charges
- Cahier de recette
- Dossier de conception

2. Guide de lecture

2.1. Maîtrise d'œuvre

La maîtrise d'œuvre rassemble l'ensemble des personnes travaillant à la réalisation du projet. Elle devra impérativement maîtriser les concepts de base du langage de programmation. Elle doit aussi pouvoir décrire les différents modules réalisés pour le programme *(cf partie 4 et 5)*, ainsi que tous ceux qui le relie (*cf partie 6,7 et 8*), en détaillant afin que le client puisse comprendre de manière claire et nette.

Pour ce projet, trois maîtres d'œuvre ont été désignés: Xie Rolland, Rayan Al Mohaize, Devasenaradjounayagar Damien.

Chacun des maîtres d'œuvre se verra être chef de projet durant 3 semaines consécutives avant de céder sa place à un autre. Le premier chef de projet sera Devasenaradjounayagar Damien, puis Rayan Al Mohaize, et enfin Xie Rolland.

Les maîtres d'œuvre sont encadrés et guidés par la maîtrise d'ouvrage. Elle est représentée par le responsable de projet : M. Wendling Laurent.

2.2. Maîtrise d'ouvrage

Le maître d'ouvrage est le client ayant commandité le projet. Les clients doivent pouvoir comprendre au mieux les modules ainsi que les variables qui leur ont été associées. Pour cela, la lecture de ce document doit leur permettre de comprendre l'utilité et le rôle de chacun des modules utilisés pour l'élaboration du projet. (*Cf. partie 4 et 5).*

La personne bénéficiant des résultats du projet est M. Wendling, qui est donc le maître d'ouvrage. M. Wendling sera également notre encadrant tout au long de la réalisation du projet. Le rôle de l'encadrant est de guider le groupe et de vérifier le bon déroulement du projet, tout en donnant en parallèle des conseils et directives. Une réunion hebdomadaire est prévue pour cela chaque vendredi à 19h00.

3. Concepts de base

Les concepts clés liés aux **Relations spatiales**, en rapport avec la **documentation interne**, incluent les:

MODULE[1] : Un module désigne originellement un fichier de code de programmation ou un fichier de bibliothèque statique ou dynamique.

VARIABLE EXTERNE[2] : une variable déclarée à l'extérieur du corps de toute fonction ou classe, et pouvant donc être utilisée n'importe où dans le programme.

4. Relation d'utilisation

4.1. Lecture des arguments pour la ligne de commande avec arguments

```
{
    //Tests pour le lancement en ligne de commande.
    //On vérifie si la méthode est appelée avec trois arguments.
    if (args.length==3)
    {
        try {
            //On vérifie si le premier argument est un .png.
            if (!args[0].endsWith(".png"))
            {
                throw new Exception("Format de fichier incompatible.");
            }
            //On vérifie si le deuxième argument vaut 1 ou 2 pour le choix de l'objet.
            if (Integer.parseInt(args[1]) != 1 && Integer.parseInt(args[1]) != 2)
            {
                throw new Exception("Le choix doit être 1 ou 2.");
            }
            //On vérifie si le troisième argument est compris entre 0 et 360
            if (Integer.parseInt(args[2]) < 0 || Integer.parseInt(args[2]) > 360)
            {
                throw new Exception("L'angle doit être compris entre 0 et 360 inclus.");
            }

        } catch (Exception e) {
            //On vérifie si une exception est levée. Si oui, affiche un message d'erreur et quitte l'application.
            System.out.println("Erreur : " + e.getMessage());
            System.exit( status: 0);
        }
    }
```

Image 4.1

Cette partie du code vise à lire les arguments pour le lancement du programme en ligne de commande, par la suite on sautera sur les choix utilisateurs sans sauter les affichages pratiques puisqu'on met directement les arguments, et donc la ligne de commande ne proposera pas les choix à faire.

Chaque argument sera utilisé, dans la partie 4.2 pour lire l'image et la partie 4.7 pour les traitements.

Dans la partie 4.8 on sort du programme avant de boucler les choix, si on utilise les arguments de cette partie.

4.2. Lecture de l'image

```
try {
    //Lecture de la photo via la sélection de l'utilisateur.
    BufferedImage image = null;
    String nomImage = null;

    //Version ligne de commande.
    if (args.length != 0)
    {
        File file = new File(args[0]);

        if (!file.exists() || file.isDirectory())
        {
            System.out.println("Le fichier spécifié n'existe pas.");
            System.exit( status: 0);
        }

        //Lecture du fichier.
        try {
            image = ImageIO.read(file);
            String fileName = file.getName();
            int dotIndex = fileName.lastIndexOf( ch: '.');
            nomImage = (dotIndex > 0) ? fileName.substring(0, dotIndex) : fileName;
        } catch (IOException e) {
            System.out.println("Erreur lors de la lecture du fichier.");
            System.exit( status: 0);
        }
    }
```

Image 4.2 partie 1

Cette partie a pour but de récupérer une image qui est donnée par l'utilisateur via la ligne de commande.

Le nom sera utilisé pour les réutiliser dans la partie 4.7 pour la sauvegarde de l'objet traité afin d'influer sur le nom.

```java
else {
    //Version GUI.
    JFileChooser fileChooser = new JFileChooser();
    int returnValue = fileChooser.showOpenDialog( parent: null);
    if (returnValue == JFileChooser.APPROVE_OPTION) {
        File file = fileChooser.getSelectedFile();

        //Lecture du fichier.(De type .png.)
        boolean fichierInvalide = true;
        while (fichierInvalide) {
            try {
                image = ImageIO.read(file);
                String fileName = file.getName();
                if (fileName.endsWith(".png")) {
                    int dotIndex = fileName.lastIndexOf( ch: '.');
                    nomImage = (dotIndex > 0) ? fileName.substring(0, dotIndex) : fileName;
                    fichierInvalide = false;
                    // Vérification des couleurs des pixels de l'image
                    //On parcourt tous les pixels
                    for (int x = 0; x < image.getWidth(); x++) {
                        for (int y = 0; y < image.getHeight(); y++) {
                            int couleur = image.getRGB(x, y);
                            //Si le pixel n'est ni jaune, ni vert, ni violet on affiche un message d'erreur
                            if (couleur != Couleurs.JAUNE && couleur != Couleurs.VERT && couleur != Couleurs.VIOLET) {
                                System.out.println("Erreur : le fichier doit contenir uniquement des pixels de couleur jaune, vert ou violet.");
                                System.exit( status: 0);
                            }
                        }
                    }
                } else {
                    System.out.println("Le fichier doit être un fichier PNG.");
                    returnValue = fileChooser.showOpenDialog( parent: null);
                    if (returnValue == JFileChooser.APPROVE_OPTION) {
                        file = fileChooser.getSelectedFile();
                    } else {
                        System.out.println("Fichier inexistant.");
                        System.exit( status: 0);
                    }
                }

            } catch (IOException e) {
                System.out.println("Erreur lors de la lecture du fichier.");
                System.exit( status: 0);
```

Image 4.2 partie 2

Cette partie a pour but de récupérer une image qui est donnée par l'utilisateur via une GUI spécifique.

Le nom sera utilisé pour les réutiliser dans la partie 4.7 pour la sauvegarde de l'objet traité afin d'influer sur le nom.

4.3. Conversion de l'image en tableaux a 2 dimensions

```
//Récupération des dimensions de la photo.
int width = image.getWidth();
int height = image.getHeight();
Pixel[][] espaceV = new Pixel[width][height];//Enregistrement de la photo sous forme d'une matrice de points.
int[] mainColors = new int[3];//Tableau des 3 différentes couleurs dans la photo.
int colorIndex = 0;

//Parcours toute l'image et stocker l'image dans une classe pixel qu'on sauvegarde dans notre matrice espaceV.
for (int y = 0; y < height; y++)
{
    for (int x = 0; x < width; x++)
    {
        int color = image.getRGB(x, y);//Sauvegarde la couleur(en int) du pixel.
        espaceV[x][y] = new Pixel(x, y, color);//Sauvegarde le pixel dans le tableau.

        //Création du tableau exhaustif de toutes les couleurs
        //Vérifie si la couleur est dans mainColors.
        boolean isInMainColors = false;
        for (int i = 0; i < mainColors.length; i++)
        {
            if (mainColors[i] == color)//On arrête la boucle si la couleur est déjà présente.
            {
                isInMainColors = true;
                break;
            }
        }
        //On ajoute la couleur dans mainColors si la couleur n'est pas dedans.
        if (!isInMainColors && colorIndex < mainColors.length) {mainColors[colorIndex++] = color;}

    }
}
```

Image 4.3

Cette partie a pour but de convertir notre image en un tableau de pixels afin d'extraire les 2 Objets dans la partie 4.4 qu'on utilisera pour les parties restantes.

4.4. Création des 2 Objets

```
//Creation des 2 tableaux pour l'objet A et B.
Objet objetA = new Objet();
objetA.addColor(Couleurs.JAUNE);

Objet objetB = new Objet();
objetB.addColor(Couleurs.VERT);

for (int x = 0; x < espaceV.length; x++)
{
    for (int y = 0; y < espaceV[0].length; y++)
    {

        int couleur = espaceV[x][y].getColor();
        Pixel pixel = espaceV[x][y];

        if (couleur == Couleurs.JAUNE)
        {
            objetA.addPoint(pixel);
        } else if (couleur == Couleurs.VERT)
        {
            objetB.addPoint(pixel);
        }
    }
}
```

Image 4.4

Cette partie a pour but de créer nos 2 objets par rapport à nos constantes “JAUNE” et “VERT” via notre tableau de pixels.

À noter que grâce à cette utilisation, il suffit de modifier les valeurs dans la classe Couleurs pour moduler le programme.

Ces objets seront utilisés pour la partie 4.5 et les parties suivantes.

4.5. Calculs

```java
//Calcul de tous les angles de l'objet A et B.
objetA.calculAngles(objetB);
objetB.calculAngles(objetA);

//Calcul des centres de gravité.
objetA.calculCentreGrav();
objetB.calculCentreGrav();
```

Image 4.5

Cette partie a pour but d'utiliser nos méthodes de calcul sur les 2 objets, pour les informations relatives aux Objets et leurs Méthodes, veuillez vous référer à la Javadoc et la partie 6.

4.6. Affichages et choix

```java
//Affiche les 2 objets d'affilée.
objetA.afficherObjet(width, height);
objetB.afficherObjet(width, height);
//Affiche les 2 couleurs
System.out.println("1. Objet jaune");
System.out.println("2. Objet vert");

//Demande à l'utilisateur le choix entre l'objet jaune ou l'objet vert.
boolean continuer = true;
//Réitération de la boucle while tant que l'utilisateur choisi la valeur 'o' ou 'O' pour choixContinuer et que la valeur booléenne continuer vaut true.
while(continuer) {
    int choixCouleur;
    if (args.length != 0) //Version ligne de commande.
    {
        choixCouleur = Integer.parseInt(args[1]);
    }
    else //Version GUI.
    {
        System.out.println("Choisissez un objet (1 ou 2) :");
        //Le choix doit obligatoirement être 1 ou 2 pour la couleur : 1 = jaune, 2 = vert.
        while (true) {
            Scanner sc1 = new Scanner(System.in);
            if (sc1.hasNextInt()) {
                choixCouleur = sc1.nextInt();
                if (choixCouleur == 1 || choixCouleur == 2) {
                    break;
                }
            } else {
                sc1.next();//Permet de traiter tous les cas d'erreur. (Saisi d'un mauvais entier, d'un caractère ou d'une chaine de caractère.)
            }
            System.out.println("\n");
            System.out.println("Veuillez entrer un entier valide!\n");
            System.out.println("Choisissez un objet (1 ou 2) :");
        }
    }
```

Image 4.6 partie 1

```java
if(choixCouleur==1) {
    System.out.println("Vous avez choisi l'Objet Jaune.");
    System.out.println();
}
else{
    System.out.println("Vous avez choisi l'Objet Vert.");
    System.out.println();
}

//Demande à l'utilisateur le choix de méthode.
//L'entier entré doit obligatoirement être entre 0 et 360 pour le choix de la méthode : 0 = barycentre ou [1,360] = paires de points.
int choixMethode;
if (args.length != 0) //Version ligne de commande.
{
    choixMethode = Integer.parseInt(args[2]);
}
else //Version GUI.
{
    Scanner sc2 = new Scanner(System.in);
    System.out.println("Entrez 0 pour la méthode du barycentre, ou choisissez le Pas de l'histogramme pour la méthode des paires de points(Entre 1 et 360):");
    //Le choix doit obligatoirement être 1 ou 2 pour la couleur : 1 = jaune, 2 = vert.
    while (true) {
        if (sc2.hasNextInt()) {
            choixMethode = sc2.nextInt();
            if (choixMethode == 0 || ((choixMethode < 361) && (choixMethode > 0))) {
                break;
            }
        } else {
            sc2.next();
        }
        System.out.println("\n");
        System.out.println("Veuillez entrer un entier valide! (0 ou un nombre entre 1 et 360.)\n");
    }
}
```

Image 4.6 partie 2

Cette partie a pour but d'afficher et sauvegarder les choix de l'utilisateur afin de les traiter à la partie 4.7.

À noter que pour le lancement via la ligne de commande avec arguments on sautera les étapes de choix car on les aura déjà.

4.7. Traitements

```java
//Traitement du choix de l'utilisateur.
if (choixCouleur == 1) //Objet jaune.
{
    //Création et sauvegarde de l'histogramme.
    objetA.creationHistogramme();
    //Traitement du barycentre.
    objetA.addAngleBarycentre(objetA.retourCentreGravite().getAnglesBarycentre(objetB));

    if(choixMethode == 0) //Choix de la méthode du barycentre.
    {
        String ref = new String( original: "Objet Vert");
        String arg = new String( original: "Objet Jaune");
        //Affichage du résultat dans le dossier Results.
        objetA.affichageRelatifBarycentre(ref,arg);
    }
    else //Choix de la méthode des paires de points.
    {
        String ref = new String( original: "Objet Vert");
        String arg = new String( original: "Objet Jaune");
        //Affichage du résultat dans le dossier Results.
        objetA.affichageRelatifHistogramme(ref,arg);
    }
    objetA.sauvegarde(nomImage,choixMethode);
    System.out.println("Les résultats pour l'objet Jaune ont été sauvegardées dans Results/.");
```

Image 4.7 partie 1

```java
} else //Objet vert.
{
    //Création et sauvegarde de l'histogramme.
    objetB.creationHistogramme();
    //Traitement du barycentre.
    objetB.addAngleBarycentre(objetB.retourCentreGravite().getAnglesBarycentre(objetA));

    if(choixMethode == 0) //Choix de la méthode du barycentre.
    {
        String ref = new String( original: "Objet Jaune");
        String arg = new String( original: "Objet Vert");
        //Affichage du résultat dans le dossier Results.
        objetB.affichageRelatifBarycentre(ref, arg);
    }
    else //Choix de la méthode des paires de points.
    {
        String ref = new String( original: "Objet Jaune");
        String arg = new String( original: "Objet Vert");
        //Affichage du résultat dans le dossier Results.
        objetB.affichageRelatifHistogramme(ref,arg);
    }

    objetB.sauvegarde(nomImage,choixMethode);
    System.out.println("Les résultats pour l'objet vert ont été sauvegardées dans Results/.");
}
```

Image 4.7 partie 2

Dans cette partie on va faire tous les traitements et afficher un message de fin.

Il faut savoir que les 2 objets possèdent les mêmes traitements. Ils peuvent être inversés, c'est-à-dire qu'on a la liberté d'étudier soit l'objet A, soit l'objet B.

On a le calcul du barycentre et de l'histogramme suivi de l'affichage de l'un ou l'autre selon les choix de l'utilisateur qu'il a faits soit soit dans la partie 4.1 ou la partie 4.6 suivi de la sauvegarde des résultats dans le fichier Results/.

Pour les informations relatives aux méthodes, veuillez vous référer à la Javadoc et à la Partie 6.

4.8. Boucle ou fin

```java
//On sort si on passe par la ligne de commande.
if(args.length!=0){System.exit((0));}

//Demande à l'utilisateur s'il souhaite continuer ou quitter.
System.out.println();
System.out.println("Souhaitez-vous continuer ? [O] pour Oui ou [N] pour non ?");
char choixContinuer = ' ';
while (args.length==0) {
    Scanner sc3 = new Scanner(System.in);
    String input = sc3.nextLine().trim();
    //Permet de traiter tous les cas d'erreur.(Saisi d'un mauvais caractère ou d'une chaine de caractère.)
    if (input.length() != 1 || (input.charAt(0) != 'o' && input.charAt(0) != 'O' && input.charAt(0) != 'n' && input.charAt(0) != 'N')) {
        System.out.println("Veuillez entrer un caractère valide !");
        System.out.println("Souhaitez-vous continuer ? Oui(O) ou Non(N) ?");
        continue;
    }
    choixContinuer = input.charAt(0);
    break;
}
//Si l'utilisateur saisi 'n' ou 'N' on sort de la boucle while et on quitte l'application.
if ((choixContinuer == 'n' || choixContinuer == 'N')) {
    continuer = false;
    System.out.println("\nMerci d'avoir utilisé notre application !");
}
```

Image 4.8

Le but de cette partie est de faire une boucle afin que le programme puisse se relancer automatiquement, en fonction de la réponse de l'utilisateur.

Si dans la partie 4.1 on avait des arguments, on sort de l'exécution.

5. Liste des modules

5.1. Modules externes

projet.java: Ce package est juste le package interne du projet.

java.io.File: Le module java.io.File permet de travailler avec des fichiers et des répertoires à travers des classes telles que File, qui fournit des méthodes pour créer, supprimer, modifier et lire des fichiers.

java.awt.image.BufferedImage: Le module java.awt.image.BufferedImage permet de créer et de manipuler des images en mémoire, notamment pour effectuer des opérations de traitement d'images, de conversion de formats et de manipulation de pixels.

javax.imageio.ImageIO: Le module javax.imageio.ImageIO permet de lire et d'écrire des images dans différents formats, tels que JPEG, PNG, GIF, BMP, etc. à l'aide de classes telles que ImageReader et ImageWriter.

javax.swing.*: Le module javax.swing.* fournit des classes et des composants graphiques pour créer des interfaces utilisateur dans des applications de bureau Java.

java.io.IOException: La classe java.io.IOException est utilisée pour signaler une erreur d'entrée/sortie lorsqu'un fichier ou un flux ne peut pas être ouvert ou lu.

java.util.Scanner: Le module java.util.Scanner fournit une classe pour analyser des données à partir de flux d'entrée tels que les fichiers ou les chaînes de caractères.

org.junit.jupiter.api.Assertions.*: Le module org.junit.jupiter.api.Assertions.* fournit des méthodes d'assertion pour les tests unitaires en Java, telles que assertEquals, assertTrue, etc.

java.io.*: Le module java.io.* fournit des classes pour les entrées/sorties de fichiers, telles que FileReader, FileWriter, ByteArrayInputStream, etc.

java.time.format.DateTimeFormatter: Le module java.time.format.DateTimeFormatter fournit des classes pour formater et analyser des dates et des heures dans différents formats.

java.util.ArrayList: Le module java.util.ArrayList fournit une implémentation de la structure de données de liste en Java qui permet d'ajouter, de supprimer et de modifier des éléments.

java.time.LocalDateTime: Le module java.time.LocalDateTime fournit une classe pour représenter une date et une heure sans fuseau horaire en Java.

org.junit.jupiter.api.Assertions.assertEquals: Le module org.junit.jupiter.api.Assertions.assertEquals est une méthode d'assertion qui vérifie si deux objets sont égaux dans les tests unitaires.

java.io.FileNotFoundException: La classe java.io.FileNotFoundException est utilisée pour signaler qu'un fichier n'a pas été trouvé lorsqu'il est ouvert ou lu.

5.2. Modules internes

Couleurs: Ce module permet de sauvegarder les constantes de couleurs.
Objet: Ce module permet d’enregistrer des objets via des objets de type Pixel et faire tous les calculs relatifs aux objets, notamment le barycentre et l’histogramme.
Pixel: Ce module permet de sauvegarder un point via sa position et sa couleur, il a des calculs relatifs a lui-même.

CouleursTest: Ce module sert à faire tous les tests unitaires relatifs à la classe Objet.
ObjetTest: Ce module sert à faire tous les tests unitaires relatifs à la classe Couleurs.
PixelTest: Ce module sert à faire tous les tests unitaires relatifs à la classe Pixel.

6. Définitions de types

6.1. Définitions de classes classiques

● Classe Couleur

- Description : La classe Couleur sert à définir les constantes de couleurs de nos images.

- Attributs :

1. public static final int VIOLET : Représente la couleur constante violette.
2. public static final int JAUNE : Représente la couleur constante jaune.
3. public static final int VERT : Représente la couleur constante verte.

- Méthodes :

Aucune.

● Classe Pixel

- Description : La classe Pixel à la responsabilité de retourner les couleurs ainsi que les coordonnées de chaque pixel, et permet le calcul des angles et celle du barycentre. Il affichera ainsi tous les angles calculés. Cette classe est utilisée pour les calculs d'angles entre un pixel d'un objet avec tous les autres pixels de l'autre objet.

- Attributs :

1. private int x,y : Les variables des points de la classe.
2. private int color : Les variables qui définissent une couleur de la classe.
3. private ArrayList<Double> angles : ArrayList permettant de stocker les valeurs des angles.

- Méthodes :

1. public int getX() : Retourne la valeur de X.

2. public int getY() : Retourne la valeur de Y.
3. public int getColor() : Retourne la couleur souhaitée.
4. public ArrayList<Double> getAngles() : Retourne l'ensemble des angles qui sont stockés dedans.
5. public void getAngles(Objet o) : Méthode permettant de faire le calcul d'angle entre 1 point et tout le reste des points.
6. public double getAnglesBarycentre(Objet o) : Méthode permettant de calculer l'angle du barycentre avec un objet passé en argument.
7. public void afficheAngles() : Méthode permettant l'affichage des angles calculés grâce à la méthode "getAngles(Objet o)".
8. public String toString(): Méthode qui retourne les informations du point.

● Classe Objet

- Description : La classe Objet a la responsabilité de stocker ses points dans le but de pouvoir créer un histogramme et de l'afficher. Il s'occupe également du calcul et de l'affichage du centre de gravité. Il sauvegarde tout cela dans un fichier. Cette classe est une classe assez polyvalente, elle est utilisée pour récolter les informations d'un objet et réaliser un histogramme de celui-ci.

- Attributs :
    1. private int color : Variable de la couleur des points de l'objet.
    2. private ArrayList<Pixel> ensemblesPoints = new ArrayList<Pixel>() : Liste de tous les points de l'objet.
    3. private ArrayList<ArrayList<Double>> histogramme = new ArrayList<ArrayList<Double>>() : ArrayList d'ArrayList qui va stocker pour chaque angle le nombre de points que l'objet a.
    4. private Pixel centreGravite : Variable du pixel de centre de gravité de l'objet.
    5. private double angleBarycentre : Variable de l'angle du Barycentre.

- Méthodes :

1. public void addColor(int color):Permets l'ajout d'une couleur passé en argument.
2. public void addPoint(Pixel p): Permets l'ajout d'un point passé en argument.
3. public int retourX(int position): Retourne la variable X d'un point à l'aide de sa position passée en argument.
4. public int retourY(int position): Retourne la variable Y d'un point à l'aide de sa position passée en argument.
5. public ArrayList<Pixel> retourneEnsemblePoints(): Retourne l'ensemble des points présent dans le ArrayList.
6. public int tailleEnsemblePoints(): Retourne la taille de ArrayList de l'ensemble de points.
7. public Pixel retourCentreGravite(): Retourne le centre de gravité de type Pixel.
8. public int retourCentreGraviteX(): Retourne le point X du centre de gravité.
9. public int retourCentreGraviteY(): Retourne le point Y du centre de gravité.
10. public void addAngleBarycentre(double angleBarycentre): Permets l'ajout de l'angle passé en argument en remplaçant la variable "angleBarycentre" par celle passé en argument.
11. public double retourAngleBarycentre(): Retourne la variable "angleBarycentre".
12. public void affichageBarycentre(): Permet l'affichage de l'angle du barycentre.
13. public void calculAngle(Objet B): Permet d'obtenir l'angle entre un point passé en argument avec les autres angles.
14. public void afficherObjet(int width,int height): Permets l'affichage seul d'un objet à l'aide de ces points ainsi que les variables qui sont en argument.
15. public void creationHistogramme(): Permet la création de l'histogramme via les points présents dans ArrayList.
16. public void affichageHistogramme(int pas): Affichage de l'histogramme sur la console avec le pas choisi.
17. public void calculCentreGrav(): Permets le calcul du centre de gravité d'un Objet.
18. public void afficherCentreGravite(int width,int height): Affichage du centre de gravité. Il sera affiché directement dans une fenêtre.
19. public void sauvegarde(String nomImage, int pas): Permet la sauvegarde de l'histogramme ou de l'angle du barycentre, dans un fichier qui sera créé automatiquement, afin de mieux visualiser et d'avoir des traces du résultat.
20. public void affichageRelatif(Objet o): Affiche la position relative d'un objet par rapport à l'autre selon les 4 directions différentes.
21. public void affichageRelatifBarycentre(String ref, String arg): Affichage complet de l'interprétation des résultats via la méthode du Barycentre.

22. public void affichageRelatifHistogramme(String ref, String arg) : Affichage complet de l'interprétation des résultats via la méthode des paires de points.
23. public void calculAngles(Objet B) : Calcul les angles de tous les points de l'objet par rapport à un autre objet passé en argument.
24. public BufferedImage createObjetImage(int width, int height) : Affiche l'objet seul à partir de ses points.
25. public void getAngleBot() : Obtenir tous les angles du haut de l'histogramme.
26. public public void getAngleLeft() : Obtenir tous les angles présents à gauche de l'histogramme.
27. public void getAngleRight() : Obtenir tous les angles présents à droite de l'histogramme.
28. public public void getAngleTop() : Obtenir tous les angles du bas de l'histogramme.
29. public int getColor() : Obtenir l'information de la variable color de l'objet.
30. ArrayList<ArrayList<Double>> getHistogramme() : Renvoie l'histogramme sous forme de ArrayList.
31. public double retourAngleBarycentre() : Donne la valeur de l'angle du barycentre de l'objet.
32. public Pixel retourCentreGravite() : Retourne le centre de gravité de type Pixel.
33. public int retourCentreGraviteX() : Retourne le point X du centre de gravité.
34. public int retourCentreGraviteY() : Retourne le point Y du centre de gravité.
35. public void setCentreGravite(Pixel centreGraviteAttendu) : Initialise le centre de gravité avec le point qui est passé en argument.
36. public int tailleEnsemblePoints() : Retourne la taille de ArrayList de l’ensemble de points.
37. public String toString() : Retourne en String la valeur du barycentre.

6.2. Définitions de classes de test unitaire

● Classe CouleurTest

- Description : On teste si chaque couleur renvoie bien la valeur entière correspondante.

- Attributs :

Aucun.

- Méthodes :

1. public void testViolet(): Test permettant de vérifier la bonne couleur.
2. public void testJaune(): Test permettant de vérifier la bonne couleur.
3. public void testVert(): Test permettant de vérifier la bonne couleur.

● Classe PixelTest

- Description : On teste unitairement les méthodes de Pixel.

- Attributs :

Aucun.

- Méthodes :

1. void testGetX(): On vérifie qu'il retourne bien x, c'est-à-dire 10.
2. void void testGetY(): On vérifie qu'il retourne bien y, c'est-à-dire 20.
3. void testGetColor(): On vérifie qu'il retourne la bonne couleur.
4. void testGetAnglesPixel():On vérifie que la taille de l'ArrayList est correcte égale à 0.
5. void testGetAnglesObjet(): On crée un objet de pixels et ajoute l'angle au l'Arraylist puis on teste la taille.
6. void testGetAnglesBarycentre(): On vérifie le résultat de l'application de la méthode.
7. void testAfficheAngles(): On vérifie que l'affichage est correct, il est inutile de vérifier la sortie standard.

● Classe PixelObjet

- Description : On teste unitairement les méthodes de Objet.

- Attributs :

Aucun.

- Méthodes :

1. public void testAddColor(): Création d'un objet et vérification de sa couleur.
2. public void testAddPoint(): Création d'un objet et vérification d'un point qu'on ajoute.
3. public void testRetourX(): Création d'un objet et vérification de la valeur x pour les pixels de l'objet.
4. public void testRetourY(): Création d'un objet et vérification de la valeur y pour les pixels de l'objet.
5. public void testRetourneEnsemblePoints(): On crée des Pixels et ajoute l'angle au l'Arraylist puis on teste s' ils sont bien présents.
6. public void testTailleEnsemblePoints(): On vérifie la taille de l'objet après lui avoir ajouté des points.
7. public void testRetourCentreGravite(): Vérifie que la méthode retourCentreGravite() retourne le centre de gravité attendu.
8. public void testRetourCentreGraviteX():Vérifie que la méthode retourCentreGravite() retourne le centre de gravité attendu.
9. public void testRetourCentreGraviteY(): Vérifie que la méthode retourCentreGravite() retourne le centre de gravité attendu.
10. public void testAddAngleBarycentre(): Vérifie le résultat attendu par la création d'un objet.
11. public void testRetourAngleBarycentre(): Vérifie le résultat attendu par la création d'un objet.
12. public void testAffichageBarycentre(): Vérifie le message attendu.
13. public void testCalculAngles(): Vérifie le calcul d'angles pour 2 objets qu'on crée en ajoutant des points spécifiques.
14. public void testAfficherObjet(): Vérifie les valeurs de width et height.
15. void testCreationHistogramme(): Vérifie le contenu d'un histogramme qu'on crée.
16. void testCalculCentreGrav(): Vérifie le résultat du centre de gravité avec des points et objets que l'on crée.
17. void testAfficherCentreGravite(): On vérifie si la fenêtre se ferme bien après son ouverture.

18.void testSauvegarde(): On crée un fichier test et on vérifie son nom, on le supprime après le test.
19.void testGetAngleBot(): On vérifie qu'il s'agit bien du résultat attendu.
20.void testGetAngleTop(): On vérifie qu'il s'agit bien du résultat attendu.
21.void testGetAngleRight(): On vérifie qu'il s'agit bien du résultat attendu.
22.void testGetAngleLeft(): On vérifie qu'il s'agit bien du résultat attendu.

7. Procédures externes

Toutes les méthodes sont visibles dans leurs Objets effectifs. En raison de leur définition “private”, ils sont seulement visibles dans notre module.

Notons d’ailleurs qu’en JAVA, il y a plusieurs types de visibilité pour les membres de la classe:

Visibilité des membres

private : ne sera visible qu'à l'intérieur d'une classe.

public : sera visible partout (membres dédiés au monde extérieur).

friendly : ne sera visible qu'aux classes du même package.

protected : ne sera visible qu'à l'intérieur d'une classe, mais peut être manipulé par les méthodes des classes héritières (indépendamment du package).

8. Variables externes

Toutes les méthodes sont visibles dans leurs Objets effectifs. En raison de leur définition “private”, ils sont seulement visibles dans notre module.

Notons d’ailleurs qu’en JAVA, il y a plusieurs types de visibilité pour les classes :

Visibilité des classes

public : sera visible partout. Il faut que la source de la classe soit dans un fichier ayant le même nom qu'elle.

friendly : ne sera visible qu'à l'intérieur du même package.

private : une classe private doit être interne à une autre.

9. Glossaires

-**Bibliothèque** : En informatique, une bibliothèque logicielle est une collection de routines, qui peuvent être déjà compilées et prêtes à être utilisées par des programmes.

-**JAVA** : Langage de programmation orienté objet.

-**Histogramme** : Graphique qui montre la répartition des données en groupes. Il aide à visualiser la distribution des valeurs dans une population ou un échantillon.
On utilisera un histogramme pour représenter les résultats des angles obtenus via la méthode des paires de points.

-**Barycentre** : Le barycentre est le point où se trouve la masse moyenne d'un objet ou d'un système de masses. C'est le point de gravité de l'objet.

-**Centre de gravité** : le centre de gravité ou CdG, appelé G, est le point d'application de la résultante des forces de gravité ou de pesanteur.

-**API** : est une interface logicielle qui permet de « connecter » un logiciel ou un service à un autre logiciel ou service afin d'échanger des données et des fonctionnalités.

-**Interface** : Une interface définit la frontière de communication entre deux entités, comme des éléments de logiciel, des composants de matériel informatique, ou un utilisateur.

-**GUI** : Une interface graphique utilisateur (ou GUI) est une interface permettant d'utiliser un PC, une tablette et d'autres appareils.

-**ligne de commande** : Une interface en ligne de commande ou ILC est une interface personne-machine dans laquelle la communication entre l'utilisateur et l'ordinateur s'effectue en mode texte : l'utilisateur tape une ligne de commande, c'est-à-dire du texte au clavier pour demander à l'ordinateur d'effectuer une opération.

-**Package** : Les packages en Java sont un mécanisme permettant d'encapsuler un groupe de classes, d'interfaces et de sous-package.

10. Références

L'utilisation de classe ainsi que des packages ont été possibles grâce à la java documentation :

https://docs.oracle.com/javase/7/docs/api/

L'introduction à la ligne de commande :
https://tutorial.djangogirls.org/fr/intro_to_command_line/

Module :https://www.jmdoudoux.fr/java/dej/chap-modules.htm

Explication de JUnit : https://www.jmdoudoux.fr/java/dej/chap-junit.htm

Explication test intégration et fonctionnel :
https://openclassrooms.com/fr/courses/6100311-testez-votre-code-java-pour-realiser-des-applications-de-qualite/6616481-decouvrez-les-tests-dintegration-et-les-tests-fonctionnels

11.Index

- Module - 1.1; 2.1; 2.2; 4.4; 5.1; 5.2; 7; 8; 11
- Barycentre - 4.7; 6.1; 6.2; 10
- Interface - 10
- Package - 5.1; 7; 8; 10
- lignes de commande - 4.1; 4.2; 4.6; 10
- Histogramme d'angle - 1.1; 4.7; 5.2; 6.1; 6.2; 10